

UNITÉ 8 · SEMAINES 13–14

Réseau: les sockets

📅 Semaines 13–14 · dates à confirmer <> Laboratoire TD7 (corrigé + défi IA) ~ 2 animations

MANUEL M8 · THÉORIE DE L'UNITÉ

Réseau: les sockets

Lecture de base de l'unité 8 (≈ 1 h 30). Les sorties sont réelles: programmes du TD7 exécutés sur la machine du cours, sur 127.0.0.1.

SOMMAIRE

- | | | | |
|----|-------------------------------------|----|--|
| §1 | Un socket est un descripteur | §4 | Modèles de serveur |
| §2 | Le cycle de vie d'une connexion TCP | §5 | Les datagrammes et le réseau d'aujourd'hui |
| §3 | Adresses, ports et noms | | |

§1 Un socket est un descripteur

Un **socket** est l'extrémité d'une communication. `socket(domain, type, protocol)` renvoie un descripteur de fichier, comme `open` ou `pipe` : on l'utilise avec `read`, `write` et `close`, il s'hérite par `fork` et occupe une entrée de la table des descripteurs. Le *domaine* indique quelles adresses il utilise: `AF_UNIX` (un chemin du système de fichiers, sur la même machine), `AF_INET` (IPv4) ou `AF_INET6` (IPv6). Le *type* indique comment il communique:

- **Connecté** (`SOCK_STREAM`, TCP sur Internet): on établit une connexion, puis un **flux d'octets** fiable et ordonné circule, comme dans un tube. C'est le téléphone de la diapositive 181.
- **Non connecté** (`SOCK_DGRAM`, UDP): chaque envoi est un **datagramme** indépendant, qui peut se perdre, se dupliquer ou arriver dans le désordre. C'est le télégramme.

La différence se mesure. Trois `write` sur TCP arrivent en un seul `read` — TCP ne sait rien des messages —, alors que sur UDP chaque `recvfrom` renvoie un datagramme entier; et si le récepteur ne lit pas à temps, UDP perd des données sans que personne reçoive d'erreur, même sur la même machine:

Sortie réelle

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3 "two", recvfrom
#3: 5 "three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8 received, 9992
lost
kernel counter Udp RcvbufErrors grew by 9992
```

Conséquence pratique: sur TCP, c'est l'application qui définit où finit chaque message (HTTP/1 utilise des lignes et `Content-Length` ; le serveur d'écho du TD7, la fin de ligne). Et comme avec les tubes, `read` et `write` peuvent être **partiels** (unité 2): `write_all` boucle jusqu'à tout envoyer.

Pratiquez → A8.1, scènes 2 et 6 · TD7 · 4

§2 Le cycle de vie d'une connexion TCP

Le **serveur** crée un socket, lui donne une adresse et un port avec `bind` et appelle `listen(fd, backlog)` : le socket devient *passif* (état LISTEN) et ne sert plus qu'à recevoir des connexions. Le **client** crée son socket et appelle `connect` avec l'adresse du serveur. Les deux noyaux font alors la **poignée de main en trois temps** : SYN du client, SYN-ACK du serveur, ACK du client. Le point clé: **c'est le noyau qui termine la poignée de main**, pas l'application. La connexion reste ESTABLISHED dans la *file d'acceptation* du socket d'écoute, et `accept` ne fait que l'en sortir et renvoyer un **nouveau descripteur** pour ce client (celui d'écoute continue d'en accepter d'autres).

Le programme `tcp_states` joue les deux rôles dans un seul processus et demande au noyau l'état de chaque socket avec `getsockopt(TCP_INFO)` :

Sortie réelle

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -          | server -
connect (no accept yet)   | listener LISTEN (queue 1/4) | client ESTABLISHED | server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED | server
ESTABLISHED
write/read                 | server read 6 bytes
client shutdown (FIN)     | listener LISTEN (queue 0/4) | client FIN_WAIT2   | server
CLOSE_WAIT
server read               | server read returns 0: end of file
server shutdown (FIN)     | listener LISTEN (queue 0/4) | client CLOSED      | server
CLOSED
                           | the client's end moved to a TIME_WAIT entry of the kernel
(60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0        0      127.0.0.1:42772 127.0.0.1:PORT
```

La **fermeture** est symétrique et se fait en deux moitiés. Celui qui appelle `close` en premier (ou `shutdown(fd, SHUT_WR)`) envoie FIN et passe par FIN_WAIT1 et FIN_WAIT2; l'autre côté reçoit le FIN, reste en CLOSE_WAIT et son `read` renvoie 0 — fin de fichier, comme avec un tube sans écrivains. Quand il ferme à son tour (LAST_ACK) et que le dernier ACK arrive, le premier côté reste en **TIME_WAIT** pendant 2-MSL (60 s sous Linux), au cas où le dernier ACK se serait perdu et que le FIN serait répété. Cette entrée survit au programme (`ss` la montre) et c'est la raison de `SO_REUSEADDR` : sans lui, relancer un serveur échoue avec `EADDRINUSE`.

Le **backlog** ne limite pas le nombre de clients que sert le serveur, mais le nombre de connexions déjà établies qui peuvent attendre `accept`. Linux en admet `backlog + 1`; avec la file pleine, il jette le SYN et le client le renvoie avec un délai exponentiel (1 s, 2 s, 4 s...):

Sortie réelle

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5, SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8, SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

🔗 Pour les curieux: les SYN cookies

Avec un backlog très petit (`./backlog 2 8`), le noyau finit par répondre avec des *SYN cookies*, une défense contre les inondations de SYN qui ne garde aucun état. Si l'ACK final arrive alors que la file d'acceptation est pleine, il est perdu: le client se croit connecté et le serveur n'en sait rien. Les compteurs de `nstat` le montrent dans le TD7 · 4. C'est pourquoi les vrais serveurs demandent un grand backlog (le plafond est `net.core.somaxconn`).

Pratiquez → A8.1 cycle de vie d'une connexion · TD7 · 1 et 4

§3 Adresses, ports et noms

Chaque domaine a sa structure d'adresse — `sockaddr_un` avec un chemin de 108 octets au plus, `sockaddr_in` avec une adresse IPv4 et un port, `sockaddr_in6` pour IPv6 — et toutes se passent à `bind`, `connect` ou `accept` comme `struct sockaddr *` avec leur taille; `struct sockaddr_storage` est assez grande pour n'importe laquelle. Les nombres dans la structure sont dans l'**ordre réseau** (*big-endian*): d'où `htons` pour le port et `htonl` pour l'adresse. L'exemple de la diapositive 187 l'oubliait, et le bogue se voit:

Sortie réelle

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000:          the kernel bound port 22555
sin_port = htons(7000):   the kernel bound port 7000
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
ip_unprivileged_port_start = 1024
sin_port = htons(80):     bind: Permission denied
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28, sockaddr_un 110, sockaddr_storage
128
```

Sur un x86-64, 7000 = 0x1b58 est stocké sous la forme des octets 58 1b; lus en *big-endian*, ils donnent 0x581b = 22555, et le serveur finit par écouter sur un autre port. La même sortie montre trois autres choses: `getaddrinfo` transforme un nom et un service (de `/etc/hosts` et `/etc/services`, ou du DNS) en une liste d'adresses IPv4 et IPv6 prêtes à l'emploi — il remplace `gethostbyname`, qui ne connaissait qu'IPv4, n'était pas sûr avec les threads et a quitté POSIX en 2008 —; les ports inférieurs à 1024 exigent des privilèges (`CAP_NET_BIND_SERVICE`); et les tailles réelles des structures.

Un socket `AF_UNIX` n'utilise ni IP ni ports: son adresse est un **fichier de type socket**. De plus, le noyau dit au serveur qui est de l'autre côté (`SO_PEERCRED`: PID et UID), ce dont se servent `systemd`, `D-Bus` ou `/var/run/docker.sock` pour décider de ce qu'ils autorisent:

Sortie réelle

```
$ ./local fso.sock
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path holds 108
bytes
the server asks the kernel who connected: pid = mine, uid 1000 (my user)
read 5 bytes; after close the file stays until unlink
still there after close: yes
```

Pratiquez → TD7 · 4, parties 5 et 6

§4 Modèles de serveur

Un serveur passe le plus clair de son temps à **attendre**: qu'un client arrive (`accept`) ou que ce client envoie quelque chose (`read`). La façon dont il organise ces attentes décide qui attend qui:

- **Itératif**: un seul thread sert un client à la fois. Si un client est lent, tous les autres attendent dans la file.
- **Un processus par client** (`fork` après `accept`): l'enfant sert et le parent revient à `accept`. Isolés les uns des autres, mais créer un processus coûte plusieurs fois plus qu'un thread (mesuré dans le TD6 · 5) et il faut éviter les zombies.

- **Un thread par client** : moins cher et avec mémoire partagée, mais 10 000 connexions feraient 10 000 threads avec leurs piles.
- **Pool de threads** : k threads créés au démarrage consomment les connexions d'une file bornée que remplit le thread principal — le producteur-consommateur de l'unité 7. Ressources bornées, mais k clients lents le saturent.
- **Boucle d'événements** : un thread demande au noyau avec `epoll_wait` quels sockets ont déjà des données et ne sert que ceux-là, si bien qu'il ne se bloque jamais sur un client lent. C'est la réponse au problème C10K et le modèle de nginx ou de Node.js; `io_uring` (Linux 5.1) pousse l'idée plus loin.

Le TD7 met en œuvre les cinq dans `http_server.c` et les mesure avec 32 clients, d'abord 1 puis 8 qui mettent 1 s à envoyer leur requête:

Sortie réelle

```
$ ./http_server -m iterative & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 981 ms, max 982 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 982 ms, max 983 ms
```

Sortie réelle

```
$ ./http_server -m pool & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 1 ms, max 1 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 981 ms, max 981 ms
```

Sortie réelle

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 1 ms, max 2 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 1 ms, max 2 ms
```

Pratiquez → A8.2 modèles de serveur · TD7 · 3 et 4

§5 Les datagrammes et le réseau d'aujourd'hui

En mode non connecté, il n'y a ni `listen` ni `accept` : le serveur lie un port connu et attend avec `recvfrom`, qui renvoie un datagramme et l'adresse de l'expéditeur ; le client utilise `sendto` avec l'adresse de destination (diapositives 194–197). On peut aussi faire `connect` sur un socket UDP pour fixer la destination et utiliser `send` / `recv`. UDP est la base du DNS, de la voix et de la vidéo en temps réel et, depuis 2021, de **QUIC** (RFC 9000), sur lequel fonctionne **HTTP/3** (RFC 9114) : la fiabilité et le chiffrement sont mis en œuvre au-dessus, dans le programme et non dans le noyau.

Deux autres changements par rapport au cours d'origine. Presque tout le trafic est chiffré avec **TLS** (https, 443) : le socket transporte des octets chiffrés et une bibliothèque (OpenSSL, GnuTLS...) fait le travail. Et les services non chiffrés de la diapositive 183 (`telnet`, `ftp`) ont été remplacés par `ssh` et https.

Pratiquez → A8.1, scène 6 (UDP) · TD7 · 4